

TERMS OF USE

=====

Document status: Active proof of concept and development state.

DevBox already has a functioning local launcher, a graphical main interface, modular structure views, master-data and documentation maintenance, documentation snapshots, local tool detection, a repository page, and initial building blocks for the DevBox-specific publication process.

The deskNode area is actively integrated into DevBox. Supervisor and daemon processes can be started and stopped from the GUI, and their console output is displayed in the deskNode log. The product version can be edited through master data. UX themes can be named, created, deleted, duplicated, renamed, and saved. Saving a theme refreshes the deskNode product data.

In the current proof of concept, deskNode has a coupled worker architecture. Tapo, FRITZ!DECT, and Shelly are discovered and monitored locally through their own venmods; supported devices process switching states and power values. The daemon synchronizes venmod data into a global device inventory, applies an initial safety read cycle, and opens the main interface only after the initial global synchronization. Structure management supports multiple building roots, one complete device pool per building, and additional additive assignments to rooms or other structure members.

Symbol management is available as a functional catalog interface. It manages deskNode device categories and consumer devices through selection menus and dialogs for creation, editing, and deletion. New devices receive one PNG source, stored under their record ID as `symbol_source_<record_id>.png`. After every successful catalog change, `create_manufacturer_db.py` is executed so `mnfctr_db.r0b` contains current theme, category, and device data.

The graphics-pack build is functional as a proof of concept. It creates prepared consumer-graphics states from symbol sources and UX themes. The current deskNode runtime already uses language, theme, and graphics data; asset selection, broader symbol coverage, and additional visual states are still being developed.

DevBox and deskNode are not finished end-user or release versions. Credential storage, long-term stability with larger device inventories, vendor-specific edge cases, testing, packaging, and cross-platform portability remain open development tasks.

First publication: Author / publisher: Markus Walloner Country: Germany (DE)

1. Subject Matter

These terms of use apply to the software or project described in the accompanying project documentation.

Short Description: DevBox is the local development hub of the CYXnTrol Development Platform. Its PySide6 interface brings together project structure, master data, documentation, local tools, product modules, repository preparation, and recurring

development workflows.

With deskNode, an active product module is integrated: a local device and structure hub for smart plugs and connected loads. deskNode is extended through replaceable venmods for supported vendors and protocols.

DevBox is not an end-user product. It is an internal working environment in which requirements, data models, UX decisions, interfaces, builds, and publication states are prepared and reviewed in a traceable way.

Long Description: DevBox combines work that would otherwise be scattered across individual scripts, folders, spreadsheets, console windows, and external applications. It manages central project and product data, provides documentation content, organizes global structure templates, detects local tools, and brings product-specific development functions together in one interface.

The first active product module is deskNode. deskNode is a local control and structure environment for smart plugs and connected loads. A supervisor starts a daemon; the daemon couples discovered venmods through a common contract and starts one worker per venmod. The current proof of concept contains local paths for Tapo, FRITZ!DECT, and Shelly. Devices are maintained in a global inventory database, while every worker writes only its own temporary runtime database. The daemon synchronizes confirmed live values, desired states, and device identities between the layers.

deskNode additionally organizes devices in independent building trees. Every building has a device pool as its complete building-wide overview and can contain spatial or functional structure members. A device can remain in a building pool and also be assigned to a room, area, desk, or other structure member. The graphical interface visualizes this structure, device and power values, and local switching states.

The DevBox page for deskNode starts and stops the supervisor and daemon, shows their console output, and maintains product versions, UX themes, and the symbol catalog. Themes are managed through named records, RGBA colors, global font files, sizes, outlines, and shape rules. Categories and consumer symbols are maintained through stable technical IDs and PNG sources. The graphics-pack build creates prepared variants for themes and states.

DevBox works locally in the project context. A launcher discovers the project root through the .root file, provides runtime data under AppData, checks external tools, and starts the GUI from a temporary working copy. The real project root remains the authoritative source for resources, databases, and function scripts. A single-instance mechanism prevents parallel DevBox windows and activates the already-open instance when started again.

The project is developed through an AI-assisted, iterative workflow. Requirements, process logic, data models, UX decisions, test cases, and acceptance are directed by the project owner; implementation steps are created, reviewed, and refined with AI-assisted development tools.

2. Terms of Use

TERMS OF USE - DRAFT VERSION

1. Scope

These terms of use apply to DevBox, the local development environment of the CYXnTrol Development Platform. DevBox is a development tool and not a finished productive end-user application.

2. Development status

DevBox and the deskNode product module integrated within it are under active development. Functions, interfaces, database structures, venmods, graphics builds, folder structures, documents, and repository processes may change, be incomplete, or contain errors. Use is at the user's own responsibility.

3. Permitted use

DevBox may be used to locally organize, document, develop, and maintain software projects owned by the user or projects the user is authorized to work on. Users are responsible for ensuring that they have the necessary rights for processed content, data, source code, images, font files, repositories, devices, and external services.

4. deskNode and device control

deskNode can, in its development state, discover local devices, read states, and switch supported loads. A switching function must not be used as a safety function, emergency stop, protective shutdown, medical control, or sole monitoring of critical infrastructure. The user remains responsible for safe configurations, local network security, physical safeguards, device compatibility, and the consequences of any switching action.

5. Credentials

Credentials for supported vendor paths are processed only locally. The current development state does not yet have a cross-platform encrypted credential vault. Until it is implemented, especially sensitive or production-critical credentials must not rely solely on this state.

6. External programs and services

DevBox may start or cooperate with locally installed programs such as Inkscape, GIMP, Git, Git Credential Manager, and installers after explicit user action. The respective providers or rights holders are responsible for those programs, their licenses, updates, data processing, and terms of use.

A repository push, graphics build, or product-specific process is prepared or started only after explicit user action. The user is responsible for target paths, repository URL, branch, visibility, commit content, and authorization to publish.

7. No warranty

To the extent permitted by law, DevBox is provided without any assurance of availability, error-free operation, fitness for a particular purpose, or complete compatibility. In particular, DevBox does not replace backups, legal review, security review, professional acceptance, or release approval.

8. Protection of the development state

DevBox uses temporary workspaces for export, graphics, runtime, and publication operations. Nevertheless, the user remains responsible for creating current backups of the project state and checking target locations, database changes, cleanup operations, Git operations, device commands, and tool starts before they are performed.

9. Documents and legal content

README, license, privacy, or terms documents generated with DevBox may contain technical templates or draft versions. Before public or commercial use, they must be reviewed and, where necessary, adapted to the specific product, its actual data flows, distribution channels, contractual relationships, and applicable law.

10. Limitation of liability

To the extent permitted by law, the author is not liable for indirect damage, data loss, lost benefits, misconfiguration, incorrect publications, faulty asset builds, faulty device switching, or consequences of using external programs and services. Mandatory legal claims remain unaffected.

11. Changes

This draft version may be adapted as DevBox evolves.

3. License Reference

License: Zero-Clause BSD License 0BSD

The complete license terms are provided in the accompanying license file.

4. Project-Specific Notes

Purpose: DevBox is intended to turn recurring and error-prone development work into traceable local tools. This includes maintaining product and manufacturer data, structured documentation, prepared publication steps, integration of local creative tools, and controlled execution of product-specific development functions.

The application provides a common working foundation for products of the CYXnTrol Development Platform. Instead of keeping workflows only as memory, folder convention, or a collection of console commands, they are retained as data, scripts, GUI functions, and verifiable process chains.

For deskNode, DevBox additionally serves as a development and configuration interface for product versions, UX themes, device categories, consumer symbols, language resources, and prepared graphics packages. The deskNode runtime is intended to receive reproducible data and prepared assets from this process. The venmod architecture should also allow new local device paths to be added without unnecessarily rebuilding the daemon or already working vendor paths.

Context: The CYXnTrol Development Platform grew out of practical development work involving many small tools, data states, graphic files, documents, and test workflows. As the number of products and functions increases, it is no longer sufficient to keep information only in individual files or in memory. A stable project root, traceable data sources, repeatable temporary workspaces, and clearly separated product modules are needed.

DevBox is the response to this need. It acts as a local development center where manufacturer and product data, documentation, global structure rules, external tools, repository preparation, and specialized product functions are brought together.

deskNode is the first product integrated as its own active DevBox area. Its purpose is the local management, visualization, and switching of supported smart plugs. Tapo, FRITZ!DECT, and Shelly are not treated as special cases embedded directly into the interface; they are connected through venmods with a shared coupler and worker contract. Names, categories, stable technical IDs, PNG sources, theme data, and pre-rendered graphics states are brought into a reproducible workflow for connected loads.

Repository Note: The DevBox repository represents the CYXnTrol Development Platform itself, not a finished end-user product. It serves as a development state, transparent work sample, and foundation for maintaining the platform.

The published state is not copied directly from the productive project root. DevBox generates a separate cleaned root_dir state. It contains intended source code, documents, and assets, while local runtime data, temporary leftovers, installers, database WAL files, and unpublished data remain outside the publication package.

The current DevBox push treats applications as a deliberate delivery window: the area is cleaned first. Afterwards, applications/deskNode/resources/scripts including contained files are copied into the publication state. __pycache__ folders and .pyc and .pyo files remain excluded. This current scope is a DevBox-specific publication process and not yet a complete deskNode product release. Before deskNode including GUI, logic, and venmods is published as its own repository or release, its export scope must be explicitly defined and tested.

DevBox is developed through an AI-assisted, iterative workflow. Requirements, architecture, data models, UX decisions, test cases, and acceptance are directed by the project owner. Implementation is created with AI-assisted development tools and checked against defined functional requirements.

Copyright (c) 2026 Markus Walloner